An approach to real-time phoneme classification for vocal sibilance correction

1st Michael Wallis
School of Electronic Eng & Computer Sci
Queen Mary University of London
London, United Kingdom
ec25253@qmul.ac.uk

2nd Johan Pauwels
School of Electronic Eng & Computer Sci
Queen Mary University of London
London, United Kingdom
j.pauwels@qmul.ac.uk

NOT PEER REVIEWED

***Abstract*—AI is continuing to push new bounds within all fields of computer science, however, there are still use cases which have not had much study performed on them. One place where there is little documentation is the integration of AI models with audio processing tools, such as VSTs or AUs, specifically on the topic of phonetic classification for de-essing of vocals. In this paper we outline the steps needed to both train and integrate an AI model into a VST for the de-essing of sung vocals. By using a simple neural network to perform sample level classification within a JUCE framework. Then taking RTNeural as an interim between the python model and C++ JUCE environment to perform classification to apply DSP which will correct the sibilance. The experiments demonstrate the functionality of this approach within a DAW showing that further development of this process can generate better state-of-the-art classification for sibilance detection for vocal processing within a real-time constant.**

***Index Terms*—Phoneme Classification, Sibilance, RTNueral, De-Essing**

I. Introduction

Sibilance are phonemes that produce high frequency content when spoken or sung. This high frequency content can be amplified by the process of being recorded and then being played back though a speaker. Human ears do not find this high frequency content to be enjoyable to listen to but removing it completely is also not possible because you would lose phonemes in the middle of words. Therefore it is the job of a mix engineer to reduce harsh sibilance while also maintaining its integrating within the audio.

Current sibilance corrective processes do not make use of AI to help with this process. The best current practice is to use a combination of high cut filters and gain reduction on the specific phonemes that are causing harsh sibilance. There is also a need for real-time sibilance control as many live venues will want to ensure a pleasant listening environment for their customers. But the speed at which sibilance happens is so quick that a sound engineer would not be able to process the sibilance on the job.

This leads to the need of an AI based, real-time detection de-esser to help correct the sibilance. As there are a few already commercially available, this paper is not looking to show the best current method. The job of this paper is to fill the missing gap in the literature as the current products have not released any documentation explaining what methods they have taken to achieve this goal. The final constraint of this paper is that the final product should be in a plug-in format, VST or AU. These plug-in formats are standard to music producers and audio engineers and should lean into that user experience.

The process of phoneme classification using AI has been well documented with high success, however the papers stop at the offline stage of processing where the time to process can be longer than the total audio length. This can then become problematic for turning these models into audio based tools to work in real time such as Speak-to-Text processors.

A simple neural network will be used for phoneme classification to ensure the high speed of audio processing is maintained. The model will look at the effects of different final classification layers on the accuracy and performance of the phoneme detection. For the real-time processing, JUCE, a C++ audio framework, will be used, which produces a VST/AU. This will allow testing in a real-world use-case. The form of DSP (Digital Signal Processing) that will be used to correct sibilance will be gain reduction and high frequency reduction, based on the processing of Mike Cave (n.d.).

Implementing audio tools also has an unseen processing restraint, sample rate. When approaching video with machine learning, a model will have to perform at minimum 24 forward processes per second, which would increase depending on frame-rate. However at the audio level with a sample rate of 16 Khz would mean the model needs to be able to perform (depending on the window size) 16000 forward process per second. The speed at which this needs to be processed would be too slow for a standard python model to be used. Audio tools are built using C++ as the DSP requires the fast processing power of C++. This means a model that works on C++ is needed. RTNeural (Chowdhury, J., 2021) solves this challenge by allowing python models to be converted and implemented into a C++ environment. RTNeural was built to allow seq-to-seq audio model changes where the input size is equal to the output size. Again this is to work at the speed of the audio thread.

II. Related Work

It is important to consider different aspects of phoneme detection and sibilance correction. For this project the current

ways that vocal correction tools correct sibilance along with what methods of phoneme detection were looked at.

A. Current vocal correction tools

There are lots of de-essers currently available that work in a real-time situation however, they realise on older styles of sibilance detection, with the standard being a form of side-chain detection. Others will state that it is always best to fix the problems at the source, so it was important to look at correct micing techniques.

1) Correct Micing Techniques: When looking at correcting sibilance within audio, most tend to state that the first place for fixing sibilance is in the recording. This method of correction can be by moving the microphone or placing a pencil to obstruct the microphone. However a study by Garner shows that this may be too reductive to the audio, eliminating the brilliance in the vocal.

> Breath and sibilance, two things that seem to be less desirable are actually the very things that make a voice sound present and close to the listener. It creates a much more intimate quality to the singer's voice. There are some producers that prefer to position the microphone above the singer, tilted down and much further away. The problem that exists with this position is two-fold. First, the microphone pre-amplifier has to be at a hotter level, which also creates a higher noise-floor, capturing more extraneaous room noise, etc. Also, the further one gets away from a condenser microphone there will be more high end and less low end frequencies. When recording a voice, it is best to try to capture as much warmth in the voice as possible. For this reason, the high and away approach for condenser microphone positioning doesn't seem to work quite as well. There is also some "presence" lost with this technique. (Garner, K. K., 2014, p.22-23)

Garner view was also supported by David Hall later in the Paper by stating that sibilance can be reduced by proper mic selection or the right mic pre amp:

> David Hall makes an interesting point that he learned his craft by watching and observing. He feels that the further we get away from that structure, we start to introduce bad practices such as distortion, sibilance that could have been minimized by proper mic selection or finding the right mic pre that matches the mic and voice. (Garner, K. K., 2014, p.116)

Fundamentally the problem of sibilance exists because of the tools used to capture audio in a digital domain. The limitation of audio engineers supply can also play a part into the generation of sibilance within audio as they are not able to provide the correct pre amp or microphone based on David's view. This means that even in the best situations, the reality of sibilance being in audio is highly probable.

2) De-Essers: De-essing is a task by which harsh sibilance - which is captured during the conversion of audio signals - is reduced to provide a more pleasing listening experience to the audio. The harshness of sibilance is due to the frequency range at which it typically is found. That range of 3 kHz to 8 kHz has been shown to be the range in which the human ear requires the least pressure to be heard (Kunchur, M. N. 2023). This idea of the ears' response effecting the listeners enjoyment of audio is not a new concept however audio companies will focus on building both microphones and speakers that capture flat frequency responses. Though this process is not perfect, the motivation is to provide as pure a sound as possible. This means the frequency range is captured correctly and at the same level of other frequencies but leads to the displeasure of listeners due to the human ear perceiving it more.

Currently there are four methods for this process of de-essing

3) Side-chain: Side-chain methods use a band filter to look specifically at the troublesome area of the frequency range (3 kHz - 8 kHz). They will then combine that EQ with a compressor so that if that frequency range has a high spike the compressor will compress the audio as a whole, in turn reducing the sibilance.

This method is effective at detection but has its limitations with correcting the signal.

4) AI (Machine Learning): AI models currently have little documentation for their method which is why this paper is looking at explaining it to more people.

The main premise is to use machine learning to classify phonemes and only apply corrective processing when we detect those phonemes.

5) Algorithm based: Companies keep algorithms a secret otherwise others would copy the process and sell it for cheaper.

The idea behind algorithm based de-essers is using the side-chain method of detection but using that detection to inform a more complex de-essing process. There can also be other methods than this but this is the simplest to give as an example.

6) Audio Separation: Audio Separation works under the idea that we can split the audio into different parts and process each part differently. This includes looking at the sibilance frequency range and making informed choices from that frequency spectrum.

7) What is being used currently: It again builds off the side chain principle of sibilance detection as that is the main way we can tell if something is sibilance or not, Using the high frequency content (Williams, P., 1986). This method helped with the reduction processing on parts of the vocal that didn't contain harsh sibilance but was not perfect in eradicating the processing on all non sibilance parts of the vocal. Further methods have been developed to improve the distinction between sibilance and no sibilance parts of audio however with the increase in AI, new methods are being looked into. See the figure below a list of current de-essing tools with their various methods outlined.

NOT PEER REVIEWED

Name	Brand	Style of De-Essing
Pro-DS (FabFilter, n.d.)	FabFilter	Algorithm based
DeEsser Plugin (Waves, n.d.)	Waves	Side-chain Measuring
Sibilance (Waves, n.d.)	Waves	Audio Separation
Oxford SuprEsser (Sonnox, n.d.)	Sonnox	Side-chain Measuring
Weiss Deess (softtube, n.d.)(Weiss, n.d.)	Softube	Multi-band Side-chain Measuring
Soothe2 (oeksound, n.d.)	Oeksound	Algorithm based
Vocal De-Esser (Antares Tech, n.d.)	AutoTune	AI (Machine Learning)
VoiceAssist (Noiseworks, n.d.)	Noiseworks	AI (Machine Learning)

TABLE I
TABLE OF TOP DE-ESSERS WITH THEIR BRAND AND STYLE OF DE-ESSING FOR COMPARISON

Another challenge with sibilance is within live audio settings. A mix engineer can take the time to cut and splice the recorded audio to remove harsh sibilance, whereas live mix engineer will have to deal with the challenges of sibilance in real time. There are two methods that can be employed for this: Firstly, doing nothing at all with sibilance. As sibilance happens at a single phone level, the time to react to it is so small that a human would not be able to predict the time needed. Second, is to have a real-time processor listening to the audio to make the changes. This real-time processing needs to be quicker than 100 ms, as speakers hearing themselves with long delays will cause speaker problems (Lee, B. S., 1950).

B. Phonetic detection

When it comes to phonetic detection with this project, there are not any key papers that we use to focus on the method of the project. Instead they were used to understand the full literature to help inform decisions for the simple neural network approach.

1) Compare other methods such as text recognition: Even with a large collection of papers and methods, Beddor (2023) explains that with the current approaches there is still a large gap within the theoretical and empirical approaches to phonetic detection. This is further supported by O'Brien, S. M. (1993) who discussed the need of combined knowledge-based and stochastic components to achieve the best results. Even with the more black-box approach of machine learning, which tends to disregard the prior knowledge aspect of approaching phonetic classification. Context within the models will help to develop smarter models.

The black box method of machine learning can misguide the outcome of some models as well, Zhou, X. et al., (2025) explored this with ASR (Automatic Speech Recognition) which typically generates the words it thinks you said, instead of what was really said. Though this inaccuracy leans into the typical hallucinations or miss classification of models. This can be amplified depending on the speaker, Williams, S. Hughes, V. and Foulkes, P. (2023) found that EAASL (English as a second language) decreased the accuracy of phonetic detection.

Classification also has other challenges within the phonetic space, such as voiceless consonants and soft phonetics which can be missed in boundary classification (Paul, B. et al., 2024)(Arias-Vergara, T. et al., 2020). Depending on the fidelity of the classification needed, more certificated approaches will need to be considered. Another problem with classification in a real world situation is the length of phonetics. Both plosives and sibilance have short intervals that require a maximum window of 10 ms to be able to determine and then process them (Stouten, V. and Van hamme, H., 2009). The shorter window times also means that models will have less samples to perform the classification with and may lead to mixed phonetics or phonetics cut short.

Even with these constraints, machine learning implementations are still able to help perform this type of classification (Lukasik, E., 2000)(Malakar, M. and Keskar, R. B., 2021).

Hestermann, S. and Deffner, N. (2020) looked at using a CNN to apply de-essing on a master track. While this process worked, it has the problem of potentially changing audio that does not need de-essing, leading to a dullness to the audio quality. This is why standard processes look at detection then processing.

2) Spoken vs Song phonetic detection: While most phonetic detection models focus upon the spoken corpus, it is not the only task within this field. The task of phonetic detection within sung vocals introduces more complex factors that models will have to learn to understand. This idea is supported by Humphrey, E. J. et al., (2019) study where they explored the difference between spoken and sung detection finding that these complex features make it harder. The complex features in sung vocals are sustained vowels, vibrato and pitch within the vocal.

The idea of pitch can be found within spoken models which look at tonal languages for detection, the same phone said with a difference in articulation can mean two different means. This is also behind audio-to-text punctuation where, in English, an upward inflection at the end of a sentence will imply a question.

Vibrato, is the modulation of pitch over a sustained period of time. Whereas pitch meant something specific for speech, vibrato should still be classified as the same phonetic as the non vibrato version of the phone. This can interfere with training models as there is not always vibrato on the whole of the phonetic but only at the start, end or middle. Specifically, if the model is not using previous samples to determine the current phone, it will lead to differences in classification.

Finally sustained vowels can be hard for models to learn as you can lose the vowel's onset shape. As the start and end of a phonetic can be part of the key aspects to classify phonetics, such as /p/ and /b/ are only determined as different from the onset of the phone. Speak corpus tend not to have these elongated vowels so specific sung corpses would need to be used during training to ensure a model is able to accurately understand them.

NOT PEER REVIEWED

III. Design

As the model did not need to beat any state of the art models, the overall design is very simple. This is intentional to give an example that can be followed for the whole process. Each part can be explored further after the the study.

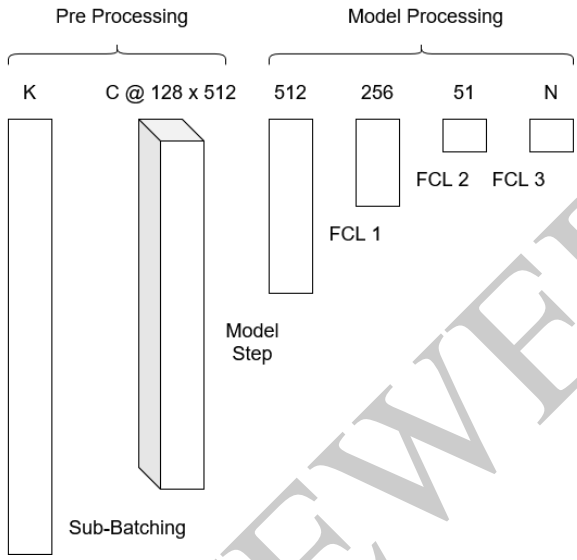


Fig. 1. Processing flow diagram for the training loop, including pre processing and the model step. Where K denotes the total sample length of any audio track, C denotes the total chunks of $[128 \times 512]$ sub-chunks generated by the sub-batching process and FCL stands for Fully Connected Layer.

A. Model Design

As the phoneme needs to be determined before the human ear can perceive a delay in processing (10ms or less) an input size of 512 was used. However at the lower sample rate of 16 Khz this window equates to 32 ms, to ensure that the correct phone will be determined in the 32 ms window, a hop size of 128 samples was implemented to give 8 ms snapshots. With higher sampling rates, this window time will decrease, leading to less perceivable processing time. For a standard CD quality recording, the 512 input size is 12 ms of delay with the hop size being 3ms (rounded).

The model is a primitive neural network with 3 fully connected layers with ReLU activation functions. The first layer starts with 512 inputs and compresses that into 256 inputs. Those 256 inputs are then compressed into 51 inputs. The final layer then converts those 51 inputs into one of the following four final classes 51, 9, 3, 2. The final classes will be one of our measurable outputs to see the effectiveness of different classes.

The final output is then passed through a log softmax to generate the log of the probabilities of the classes. The reason to use a softmax classification instead of a hardmax classification is to help with the transition of phonemes. Imagine there is an input sample of 256 of the phone /k/ which is then mixed with another 256 samples of the phone /a/ in the word /kat/ cat. The hardmax may prioritize the /a/ phone instead of the /k/ which would be in the process of being de-essed. The softmax allows the model to know that both would be present and potentially processed.

Depending on what the final class's output amount is, will also change what the model detects.

- 51 classes, All 51 ARPABET (Wikipedia contributors, 2026) phonemes including a 'sil' for silence.
- 9 classes, All 8 sibilance phonemes ch, f, k, s, sh, t, th, zh with a 'Other' tag.
- 3 classes, Grouping the sibilance into 'Air' and 'Cut' sibilance with the 'Other' class.
- 2 classes, Sibilance or not.

B. datasets

For training, the data was split 80% training with 20% validation. This split was seeded so that additional training would ensure the same audio is being passed into the training set and removing the chance of contamination. As the testing was done with an external audio source, a testing split was not needed.

All datasets used varying phonetic alphabets, however most were built upon the ARPABET version. All datasets were phased into the 51, 2 ASCII characters ARPABET classes (note that this excludes the TIMIT additional phonetics). If a phoneme was not in the 51 classes, it would be assigned to the closest match phoneme of the ARPABET alphabet.

NOT PEER REVIEWED

Dataset	No. Speakers	Sample Rate	Phonetic Alphabet
TIMIT (Garofolo, J. S. et al., 1993)	630	16000	ARPABET+ (Wikipedia contributors, 2026)
Buckeye Corups (Pitt, M. A. et al., 2007)	40	16000	Modified ARPABET
NUS-48E (Duan, Z. Fang, H. Li, B. Sim, K. C. and Wang, Y., 2013)	12	16000	ARPABET
CSD (Choi, S. Kim, W. Park, S. Yong, S. and Nam, J., 2020)	1	44100	IPA (Wikipedia contributors, 2026)

TABLE II
DATASET INFORMATION WITH COMPARISON

C. Dataloader Class

As all datasets had a different underlying structure, a custom dataloader class was created to process the datasets by their name. It would first store all audio and phonetics as location based text files to save on memory while being processed. Once called the dataloader would return both the audio and phonetics as their true formats. Librosa was used to load the audio into a one dimensional array. Each sample would then be matched with the cosponsoring phonetic which was retrieved from the phonetics file.

This would give us a 2 dimensional array the length of the audio. Where dimension one was the audio samples ranging from -1 to 1, with dimension two being the phonetic classes for each sample of the audio. This method allowed us to cut up the array into chunks while still preserving the phonetic data of each sample. Otherwise we would lose what each sample was in terms of phonetics.

For training, augmentation was also used to help with more rigorous model training. When allowed, there were four augmentations that could be applied.

1) Phase Inversion
2) High Pass Equalization (0-150 Hz cut off)
3) Low Pass Equalization (3000-18000 Hz cut off)
4) Room Reverb (0-50% mix with 0.01-4 ms delay time)

Each augmentation had a unique 40% chance of being applied to allow for more non augmented audio to be processed than augmented. The unique augmentations also allowed for simple to complex changes to the original without needing to specify each combination of effects.

The focus on augmentation effects were light but effective, such as reverb for room spill or high pass equalization to remove the fundamentals of lower voices.

D. Training

Training was performed with check pointing to ensure that the best model was saved regardless of the final model at the end of the epochs. They were trained with a learning rate of 5e-4 for 50 epochs. The Adam optimizer was used with a criterion of KL Division Loss on 'batchmean' for the softmax classification. As mentioned the model uses a log softmax layer so that another log conversion was not needed before passing into the criterion.

As the audio files were not a set size and had a large range in length, no batching function was used for the dataloaders. This means that the batch size was limited to 1. However to combat this and help with the speed of training, a batching method was used inside the training loop to emulate that of a dataloader batch.

As audio will be turned into chunks of size 512 to be passed into the model, preprocessing these 512 windows before sending them into the model allowed a batch size of 128, 512 windows to be passed through the model at once. This batch method inside the training loop was not feasible outside of the training loop and was left inside as it worked as intended.

E. JUCE Implementation

As the training was performed on python using PyTorch, being able to use the model in JUCE requires converting it into a format that is C++ readable. RTNeural (Chowdhury, J., 2021) was used to be the intermediary for this process. Any model used was turned into a JSON file which could then be read as binary code into JUCE through the RTNeural model. It is important to ensure that the model you build in JUCE has to be the same as the PyTorch model.

Inside JUCE only two controls are given to the user, a gain reduction slider ranging from 0 dB to -48 dB, and a high cut from 20 kHz to 1 kHz. Both ranges are more than what would be used on average to ensure users would be able to reduce to a level sufficient for their needs. Additional changes could be made to allow for specific phone level controls like the Vocal De-Esser (Antares Tech, n.d.) by AutoTune.

The full work flow of the JUCE code can be seen in the figure 2.

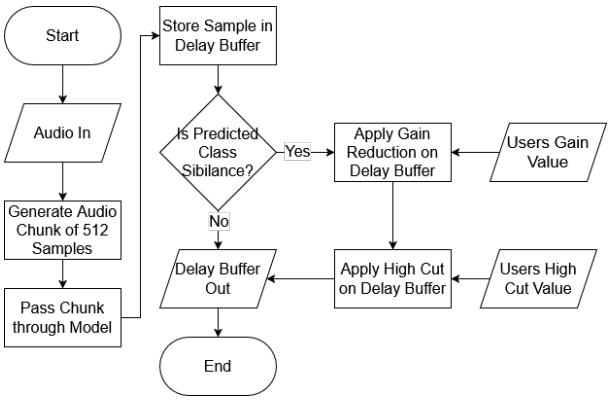


Fig. 2. Flow chart of audio processing within the JUCE processor block.

During the JUCE flow path there is a delay buffer. This stores the current 512 samples plus the old 512 samples. Using a delay like this can allow you to determine if there

NOT PEER REVIEWED

is a phoneme that needs processing in the new 512 samples, and then start applying the corrective processing to the old samples which will be sent to the user to hear. This delay of 512 samples is small enough that the human ear will not perceive the delay in audio if they were hearing both the real speaker and the process at the same time.

As the ramping of the effects also takes time to fade back to no effect, when a new 512 samples is detected as not having sibilance within it, the ramping will have reset by the time the 512 samples will then be sent to the user. This effectively processes the right phonemes with minimal delay to the user.

IV. Results

To ensure there was no data leakage for the evaluation from the datasets, an external audio source was created. The audio was recorded at 44.1 Khz with a SM7dB (SM7dB User Guide, 2023) into an Antelope Audio Zen Go Synergy Core (Antelope Audio, n.d.). The vocalist was a British male in his 20's recorded in a quiet room. As the training audio was trained on both 44.1 kHz and 16 kHz sample relates, the audio was also downsampled to 16 kHz to perform testing on both.

The test audio text was created to use the 8 sibilance phonemes [/ch/, /f/, /k/, /s/, /sh/, /t/, /th/, /zh/]. For testing a precision/recall test were carried out. There was a total of 29 phonemes that should be classed as sibilance and processing applied. This was compared to the other phonemes that should not be classified. This was calculated by using the formulas below:

$$P_Q(r) := \frac{1}{r}\sum_{k=1}^{r} x_Q(k) \tag{1}$$

$$R_Q(r) := \frac{1}{|I_Q|}\sum_{k=1}^{r} x_Q(k) \tag{2}$$

The full text that was both sung and spoken is as follows:

Its my pleasure to show you the thin Chinese cafe.
great for leisure and a place to eat on the weekday.
If we measure the best thing is the chicken satay.
Their final treasure, is what they get on a payday.

The sung vocals were sung in a light nursery rhyme style at 120 bpm in the key of C Major. Both the melody and chords that were sung to can be found in the supplementary files.

For the spoken audio, it was not spoken at a set rate but was done in a story style with accents of the rhyming words.

The total for each phoneme of sibilance is as follows:

- /ch/ - 2
- /f/ - 4
- /k/ - 3
- /s/ - 3
- /sh/ - 3
- /t/ - 1
- /th/ - 2
- /zh/ - 4

To downsample the audio to 16 kHz a librosa script was used to take the audio, load it with the sample rate and same the files with a different name to not lose the original. All the audio files were normalized using the DAWs (digital audio workstation) normalize function on export.

A. Evaluation

The first test was testing how the different final classification layers effected the precision and recall of the desired phonetics. A phonetic truth mapping was produced for each of the four classification models to be tested against. This can be found in the supplementary files.

To ensure accurate detection these tests were performed offline and recorded into a .txt document (also in the supplementary files). The audio files were all processed through the models as well to check the audio against the predictions. Tables III and IV show the results of the percussion and recall functions on the 4 audio files.

Model	Sung			
	16 kHz		44.1 kHz	
	Precision	Recall	Precision	Recall
51 Classes	*55.74%*	**87.93%**	*18.32%*	*26.42%*
9 Classes	**72.91%**	*62.64%*	*25.36%*	*17.1%*
3 Classes	*71.47%*	*64.08%*	**53.8%**	*22.52%*
2 Classes	*57.92%*	*73.56%*	*24.84%*	**27.0%**

TABLE III

THE PRECISION AND RECALL OF 4 VOCAL DE-ESSERS WITH TWO DIFFERENT SAMPLE RATES OF AUDIO PERFORMED ON SUNG VOCALS.

Model	Spoken			
	16 kHz		44.1 kHz	
	Precision	Recall	Precision	Recall
51 Classes	*55.59%*	**91.91%**	*19.31%*	**27.16%**
9 Classes	*66.11%*	*68.42%*	*21.99%*	*16.93%*
3 Classes	**68.82%**	*70.81%*	**38.29%**	*17.9%*
2 Classes	*56.42%*	*80.06%*	*20.04%*	*24.04%*

TABLE IV

THE PRECISION AND RECALL OF 4 VOCAL DE-ESSERS WITH TWO DIFFERENT SAMPLE RATES OF AUDIO PERFORMED ON SPOKEN VOCALS.

The columns with the highest precision or recall have been highlighted in bold to show which performed the best comparatively.

From the data the 51 class model shows to have the best performance for spoken audio by having the highest recall for both 16 and 44.1 kHz sample rates. However if you look at the consistency of the models in both metrics, the best looking model appears to be the 3 class model. With high precision in both spoken and sung audio clips, the differences show a stronger learning ability for the air sibilance and cut sibilance. This result also supports AutoTune's classification setup, as theirs also performs 3 classifications with [Air, Cut, or Other]. It's important to remember that the 'other' category should determine all other phonemes. There can also be the possible fourth classes of specificity detecting silence but without more information it will be unknown.

Another interesting note from this data is that the 9 class model performs equally as well as the 3 class model but only when looking at 16 kHz audio, there is a noticeable drop when used on the 44.1 kHz sample rate audio. This dip most likely

NOT PEER REVIEWED

comes from the smaller amount of 44.1 kHz dataset audio present in the training set.

Strangely the worst performing model is the 2 class model, though this may seem backwards as the AI model only has to learn two classifications. The problem comes from the scope of what those classifications are trying to detect. The model to correctly learn if there is sibilance has to learn 8 different types of phonemes and then any of the differences that speakers may have when speaking them.

This over-simplification makes it harder for the 2 class model to really 'learn' what sibilance is. Compared to the 3 class model which still has to learn 3 and 5 different sibilance, it can focus on being more accurate about what the sibilance accurately is.

The second test was to see the effectiveness of downsampling one of the datasets, specifically the Sung 44.1 kHz Children's Song Dataset. To save on processing power, a common practice of speech AI engineers is downsample higher sample rate audio into 16 kHz. 16 kHz still provides all the necessary information that the human ear needs to understand the human voice. However the downside is that high frequency content is removed due to the Nyquist theorem.

Sibilance is dominated by high frequency content specificity at the 8 kHz range, which would push the 16 kHz Nyquist theorem to the upper limit. Anything above the 8 kHz will have the potential to be aliased as a lower frequency. Thus making sibilance detection even harder.

Model	Sung			
	16 kHz		44.1 kHz	
	Precision	Recall	Precision	Recall
CSD 16000	*50.0%*	*1.15%*	*0.0%*	*0.0%*
CSD 44100	*0.0%*	*0.0%*	*0.0%*	*0.0%*

TABLE V

THE PRECISION AND RECALL OF SUNG AUDIO PERFORMED ON THE STRANDED SAMPLING RATE AND DOWNSAMPLED RATE OF THE CSD DATASET.

Model	Spoken			
	16 kHz		44.1 kHz	
	Precision	Recall	Precision	Recall
CSD 16000	*61.11%*	*3.18%*	*0.0%*	*0.0%*
CSD 44100	*0.0%*	*0.0%*	*0.0%*	*0.0%*

TABLE VI

THE PRECISION AND RECALL OF SPOKEN AUDIO PERFORMED ON THE STRANDED SAMPLING RATE AND DOWNSAMPLED RATE OF THE CSD DATASET.

The results for this test were underwhelming as the models seemed to have over fit to only detect 'other' and not sibilance. As the CSD only has one vocal which is a female, the models most likely did not understand the male vocal that was passed through.

Though we only have results for the CSD 16 kHz model on the 16 kHz sample rate audio, we can see that it was still able to perform some part of the detection with the lower quality audio. However, the comparison of the 2 class model from tables V and VI with the one trained on all datasets, shows that this smaller model was vastly underperforming.

B. Audio Comparisons

All audio was passed through the models and recorded with two parts. A processed file, which was the output of the VST on the audio, along with a mask file. The mask file is the original audio played back with the processed file but the processed file has been phase inverted. This causes the waves to create deconstructive interference and you only hear the difference between the two audio files. This mask file then becomes everything removed from the original.

The audio was also passed through the AutoTune De-Esser and was recorded using the same process. As no automation detection from the VST could be created, the use of the AutoTune for precision and recall was not used.

As audio quality is subjective, all audio files have been added into the supplementary materials folder for anyone to listen to for their own opinions on the outputs of the different models. From the authors perspective the 9 classification model has the best overall sound out of the 5 trained models. It is comparable to that of the output from the AutoTune VST but is not polished in quality.

It should be noted that the DAW that was used did not go below a sampling rate of 22050 Hz which means the exports of the 16 kHz audio files will come out at 44.1 kHz audio. The audio is still showing the correct de-essing.

V. Future Work

With the limitation on how many papers have been published on this type of processing. This topic has lots of other paths to explore. Here are a few of the other aspects others may choose to research with more time.

A. Improvements on the neural network

There are already more advanced neural networks for phoneme classification which have not been tried in this set up. A good starting place will be that of a RNN (recurrent neural network), these models use a time based attention to help inform the current decision from the model using the previous outputs from the model. This is beneficial to this methodology as each new hop can use the understanding of the last to see if the model has just had sibilance to inform if the next phoneme will still have sibilance.

Other neural networks should also be considered such as mutli-step paths. E.g. using a FFT (fast Fourier transform) to get a spectrogram of the audio which can then be passed through a convolutional network. The limitation with this type of neural network is the processing speed to apply each step. As converting audio into different formats creates more processes, a limit will be reached on how much you can do before the next audio chunk is needed.

Through a combination of these approaches, new neural networks can be derived to help build upon this challenge of real time phoneme classification.

B. Sample Rate

While this paper attempted to look at the effects of sample rate on the effectiveness of the phoneme classification. The

NOT PEER REVIEWED

test errors would require more testing to provide real results on the usefulness of different sample rates.

There are two main ways to approach the sample rate problem. Firstly finding a sample rate that accurately maintains the key information needed for the sibilance detection. Or secondly, testing the effectiveness of multi-sample rate models. While the first method will be easier to test and get results back, the second method should not be redundant as having the multi sample rate models could better work with warps in vocal tracks, such as time stretching. Simply down sampling could lead to the Nyquist theorem causing aliasing that struggles to accurately represent the phoneme that should be heard.

C. Greater Control

The last suggestion for further work is the user controls. As this project only looked at implementing the basic functions only two controls were given to the user, high cut and gain reduction. Attack for both of these controls plus a threshold selector would be additions worth looking at. They will give more control over the shape of the de-essing while also not over burdening the user's experience with too many controls.

VI. Conclusion

The use of AI classification within audio plug-ins is plausible and straight forward. Though this paper does not explore in detail the nuances of building such devices, the effect of using a simple neural network has results that show there are better outputs for that model. This demonstrates the need for further exploration into this field.

Another key aspect which has been shown to work is the real-time capabilities of the device. While running audio through then there were no technical problems or large CPU spikes that would cause problems if the device was used in a live setting. Though not fully tested in a live situation the short delay buffer time also demonstrated that it worked correctly during the recording of audio, meaning that the delay would not be perceivable in the room. This is a great success for the device as taking it now offline will become a lot simpler.

One important take away from the results was that the 3 class classification plays a better role at detecting the sibilance that needs to be processed more so than any other outputs. Though the 9 classes were also comparable on 16 kHz sample rate audio, it did have its limitations.

Coming back to this project with a better neural network and better understanding of how the sample rate affects the classification of the model would set a strong bar for the state-of-the-art for real-time, phonetics vocal correction.

Acknowledgment

Thank you to my supervisor Johan Pauwels for the guidance on this project, including the many meetings that we had to ensure that it was correct and accurate.

References

[1] Antares Tech (n.d.) "Vocal De-Esser". Available at: https://www.antarestech.com/products/ai-powered-vocal-chain/vocal-de-esser/overview. (Accessed: 2026-06-01).

[2] Antelope Audio, (n.d.) "Zen Go Synergy Core user manual". Available at: https://atmosphere82.attachments9.freshdesk.com/data/helpdesk/attachments/production/42101679472/original/EN%20-%20Zen%20Go%20SC%20Manual.pdf. (Accessed: 2026-07-06).

[3] Arias-Vergara, T. et al., (2020) "Automatic detection of voice onset time in voiceless plosives using gated recurrent units". Digital Signal Processing, vol. 104, p. 102779. Available at: https://www.sciencedirect.com/science/article/pii/S105120042030124X. (Accessed: 2026-06-01).

[4] Beddor, P. S. (2023) "Advancements of phonetics in the 21st century: Theoretical and empirical issues in the phonetics of sound change". Journal of Phonetics, vol. 98, p. 101228. Available at: https://www.sciencedirect.com/science/article/pii/S0095447023000177. (Accessed: 2026-06-01).

[5] Cave, M. (n.d.) Mike Cave, Available at: https://mikecave.co.uk/biography/.

[6] Choi, S. Kim, W. Park, S. Yong, S. and Nam, J. (2020) "Children's song dataset for singing voice research," in Extended Abstracts for the Late-Breaking Demo Session of the 21st International Society for Music Information Retrieval Conference (ISMIR), Montr´eal, Canada, 2020. Available at: https://program.ismir2020.net/static/lbd/ISMIR2020-LBD-435-abstract.pdf. (Accessed: 2026-06-01).

[7] Chowdhury, J. (2021) "RTNeural: fast neural inferencing for real-time systems". arXiv preprint, arXiv:2106.03037.

[8] Duan, Z. Fang, H. Li, B. Sim, K. C. and Wang, Y. (2013) "The NUS sung and spoken lyrics corpus: A quantitative comparison of singing and speech". In Proceedings of the 2013 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC). IEEE, 2013, pp. 1–9. Available at: https://smcnus.comp.nus.edu.sg/archive/pdf/2012-2013/2013 05-Pub-NUS-48E.pdf. (Accessed: 2026-06-01).

[9] FabFilter (n.d.) "Pro-DS de-esser plug-in". Available at: https://www.fabfilter.com/products/pro-ds-de-esser-plug-in. (Accessed: 2026-06-01).

[10] Garner, K. K. (2014) "Vocal recording techniques for the modern digital studio". Doctoral essay, University of Miami, Coral Gables, FL. Available at: https://scholarship.miami.edu/esploro/outputs/doctoral/Vocal-Recording-Techniques-for-the-Modern/991031447873002976. (Accessed: 2026-06-01).

[11] Garofolo, J. S. et al., (1993) "TIMIT acoustic-phonetic continuous speech corpus LDC93S1". Web Download, Philadelphia, available from Linguistic Data Consortium.

[12] Hestermann, S. and Deffner, N. (2020) "Enhanced de-essing via neural networks". in Machine Learning and Knowledge Discovery in Databases, P. Cellier and K. Driessens, Eds. Cham: Springer International Publishing, pp. 537–542. (Accessed: 2026-06-01).

[13] Humphrey, E. J. et al., (2019) "An introduction to signal processing for singing-voice analysis: High notes in the effort to automate the understanding of vocals in music". IEEE Signal Processing Magazine, vol. 36, no. 1, pp.82–94. (Accessed: 2026-06-01).

[14] Kunchur, M. N. (2023) "The human auditory system and audio". Available at: https://arxiv.org/abs/2307.00084. (Accessed: 2026-07-01).

[15] Lee, B. S. (1950) "Effects of delayed speech feedback". The Journal of the Acoustical Society of America, vol. 22, no. 6, pp. 824–826. (Accessed: 2026-07-02).

[16] Lukasik, E. (2000) "Classification of voiceless plosives using wavelet packet based approaches". in 2000 10th European Signal Processing Conference, pp. 1–4. (Accessed: 2026-06-01).

[17] Malakar, M. and Keskar, R. B. (2021) "Progress of machine learning based automatic phoneme recognition and its prospect". Speech Communication, vol. 135, pp. 37–53. Available at: https://www.sciencedirect.com/science/article/pii/S0167639321001084. (Accessed: 2026-06-01).

[18] Noiseworks (n.d.) "VoiceAssist ai vocal cleanup plugin with ara integration". Available at: https://noiseworksaudio.com/products/voiceassist/. (Accessed: 2026-06-01).

[19] Noiseworks (n.d.) "VoiceAssist manual". Available at: https://paddle.s3.amazonaws.com/fulfillment_downloads/

NOT PEER REVIEWED

162412/918576/51A5eo5QjuhDdWwtP8mD_VoiceAssist_Manual 2026%3A04.pdf. (Accessed: 2026-06-01).
[20] oeksound (n.d.) “soothe2 manual”. Available at: https://oeksound.com/manuals/soothe2/. (Accessed: 2026-06-01).
[21] O’Brien, S. M. (1993) “Spectral features of plosives in connected-speech signals”. International Journal of Man-Machine Studies, vol. 38, no. 1, pp. 97–127. Available at: https://www.sciencedirect.com/science/article/pii/S0020737383710060. (Accessed: 2026-06-01).
[22] Paul, B. et al., (2024) “Spoken word recognition using a novel speech boundary segment of voiceless articulatory consonants”. International Journal of Information Technology, vol. 16, no. 4, pp. 2661–2673. Available at: https://doi.org/10.1007/s41870-024-01776-3. (Accessed: 2026-06-01).
[23] Pitt, M. A. et al., (2007) “Buckeye corpus of conversational speech (2nd release)”. Available at: http://www.buckeyecorpus.osu.edu, Columbus, OH, distributor.
[24] softtube (n.d.) “Weiss Deess”. Available at: https://www.softube.com/uk/user-manuals/weiss-deess. (Accessed: 2026-06-01).
[25] SM7dB User Guide, (2023) Shure Incorporated. Available at: https://www.shure.com/en-US/docs/guide/SM7dB. (Accessed: 2026-07-06).
[26] Sonnox (n.d.) “Oxford SuprEsser user guide”. Available at: https://dload.sonnoxplugins.com/pub/plugins/UserGuides/Oxford%20SuprEsser%20User%20Guide.html. (Accessed: 2026-06-01).
[27] Stouten, V. and Van hamme, H. (2009) “Automatic voice onset time estimation from reassignment spectra”. Speech Communication, vol. 51, no. 12, pp. 1194–1205. Available at: https://www.sciencedirect.com/science/article/pii/S0167639309000892. (Accessed: 2026-06-01).
[28] Waves (n.d.) “DeEsser user manual”. Available at: https://assets.wavescdn.com/pdf/plugins/deesser.pdf. (Accessed: 2026-06-01).
[29] Waves (n.d.) “Sibilance deesser user guide”. Available at: https://assets.wavescdn.com/pdf/plugins/sibilance.pdf. (Accessed: 2026-06-01).
[30] Weiss (n.d.) “GAMBIT DS1-MK3 operating manual”. Available at: https://weiss.ch/wp-content/uploads/2022/07/DS1-MK3-Manual.pdf. (Accessed: 2026-06-01).
[31] Wikipedia contributors, (2026) “Arpabet — Wikipedia, the free encyclopedia”. Available at: https://en.wikipedia.org/w/index.php?title=ARPABET&oldid=1361557644. (Accessed: 2026-06-01).
[32] Wikipedia contributors, (2026) “International phonetic alphabet —Wikipedia, the free encyclopedia”. Available at: https://en.wikipedia.org/w/index.php?title=International Phonetic Alphabet&oldid=1361989650. (Accessed: 2026-06-01).
[33] Williams, P. (1986) “De-esser project”. Home & Studio Recording. Available at: https://www.muzines.co.uk/articles/de-esser-project/12857. (Accessed: 2026-06-01).
[34] Williams, S. Hughes, V. and Foulkes. P. (2023) “Acoustic analysis of l2 english phonemes for language identification”. in International Congress of Phonetic Sciences. Available at: https://api.semanticscholar.org/CorpusID:265154043. (Accessed: 2026-06-01).
[35] Zhou, X. et al., (2025) “Towards accurate phonetic error detection through phoneme similarity modeling”. Available at: https://arxiv.org/abs/2507.14346. (Accessed: 2026-06-01).

NOT PEER REVIEWED